

**LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 3: BLADE TEMPLATE LARAVEL**



**Universitas
Telkom**

Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

**UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026**

1. Tujuan

Adapun tujuan yang ingin dicapai pada praktikum tugas Modul 3 ini adalah sebagai berikut:

1. Membangun sistem pelaporan banjir sederhana bernama LaporBanjir dengan pola MVC Laravel tanpa memakai database.
2. Menerapkan layout utama memakai `@extends`, `@section`, dan `@yield` supaya header, navigasi, dan footer cukup ditulis satu kali untuk semua halaman.
3. Memakai directive percabangan `@if`, `@elseif`, dan `@else` untuk menentukan status genangan air, serta directive perulangan `@forelse` untuk menampilkan daftar laporan.
4. Membuat komponen Blade `<x-alert>` untuk menampilkan pesan laporan berhasil dikirim dan partial `@include` untuk menampilkan kartu laporan.
5. Menghubungkan route GET dan POST ke method controller lalu menguji ketiga halaman melalui browser.

2. Alat dan Bahan

Praktikum ini dikerjakan menggunakan satu buah laptop dengan prosesor Intel Core i7-13650HX dan sistem operasi Windows 11. Perangkat lunak yang digunakan meliputi Laragon sebagai lingkungan pengembangan web lokal, PHP versi 8.3.30 sebagai bahasa pemrograman sisi server, Composer versi 2.10.3 sebagai dependency manager, Laravel Framework versi 13.33.0 sebagai framework utama aplikasi, Visual Studio Code sebagai teks editor untuk menuliskan kode program, Git sebagai version control untuk mengelola kode sumber, serta Google Chrome sebagai web browser untuk menjalankan dan menguji aplikasi.

3. Langkah-Langkah Praktikum

Tugas praktikum yang saya kerjakan adalah nomor 1, yaitu sistem pelaporan banjir sederhana bernama LaporBanjir. Sistemnya terdiri atas halaman form pelaporan, halaman konfirmasi, dan halaman daftar laporan. Karena masih berupa prototipe, aplikasi tidak memakai database dan seluruh data laporan ditulis sebagai array di dalam controller. Langkah pengerjaannya diuraikan berikut ini.

3.1 Membuat Controller

Controller dibuat melalui perintah artisan berikut, yang secara otomatis menghasilkan file `app/Http/Controllers/LaporanController.php` beserta kerangka class-nya:

```
1 php artisan make:controller LaporanController
```

File controller berisi tiga method. Method `create()` mengirim view form, method `store()` menerima data yang dikirim lewat POST, dan method `index()` menyiapkan data daftar laporan. Isi method `store()` adalah sebagai berikut:

```
1 public function store(Request $request)
2 {
3     $laporan = [
4         'nama_pelapor' => $request->nama_pelapor,
5         'lokasi' => $request->lokasi,
6         'tinggi_genangan' => $request->tinggi_genangan,
7     ];
8
9     return view('laporan.konfirmasi', compact('laporan'));
10 }
```

Method `store()` mengambil tiga isian form lalu mengopernya ke view konfirmasi dengan `compact('laporan')`. Method `index()` mengisi array `$laporans` dengan empat data contoh, tiap data memuat nama pelapor, lokasi kejadian, tinggi genangan, dan waktu laporan. Tidak ada data yang disimpan ke database karena tugas ini memang meminta versi prototipe.

3.2 Mendefinisikan Route

Route didefinisikan pada file `routes/web.php` seperti berikut:

```

1 use App\Http\Controllers\LaporanController;
2 use Illuminate\Support\Facades\Route;
3
4 Route::get('/', [LaporanController::class, 'create'])->name('laporan.create');
5 Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');
6 Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');

```

Route GET / menampilkan halaman form, route POST /laporan memproses pengiriman form, dan route GET /laporan menampilkan daftar laporan. Ketiganya diberi nama lewat `->name()` supaya bisa dipanggil dari Blade dengan `route('laporan.create')` dan `route('laporan.index')`. Menu navigasi di layout memakai named route tersebut, jadi alamat URL cukup didefinisikan satu kali di `routes/web.php`.

3.3 Membuat Layout Utama

Layout utama dibuat pada file `resources/views/layouts/app.blade.php`. File ini berfungsi sebagai kerangka halaman: judul diisi lewat `@yield`, isi halaman diisi lewat `@yield('content')`, dan CSS ditulis langsung di dalam tag `style` tanpa bantuan library lain:

```

1 <title>@yield('title')</title>
2 <style>
3     body {
4         font-family: Arial, Helvetica, sans-serif;
5         background: #f3f4f6;
6     }
7     .site-header { background: #0ea5e9; color: #fff; }
8 </style>

```

```

1 <header class="site-header">
2   <a class="brand" href="{{ route('laporan.create') }}">LaporBanjir</a>
3   <nav>
4     <a href="{{ route('laporan.create') }}">Form Pelaporan</a>
5     <a href="{{ route('laporan.index') }}">Daftar Laporan</a>
6   </nav>
7 </header>
8
9 <main class="container">
10   @yield('content')
11 </main>

```

Header memuat nama sistem LaporBanjir beserta menu navigasi yang memakai named route, footer memuat nama instansi penyelenggara, dan tidak ada satu pun kode JavaScript di dalam layout ini. Setiap halaman selanjutnya cukup memanggil `@extends` layout lalu mengisi bagian section miliknya sendiri.

3.4 Membuat Halaman Form

Halaman form dibuat pada `resources/views/laporan/create.blade.php`. View ini mewarisi layout utama lalu mengisi section content dengan formulir berikut:

```

1 @extends('layouts.app')
2
3 @section('title', 'Form Pelaporan Banjir')
4
5 @section('content')
6   <h1>Form Pelaporan Banjir</h1>
7
8   <form method="POST" action="{{ route('laporan.store') }}">
9     @csrf
10
11     <label for="nama_pelapor">Nama Pelapor</label>
12     <input type="text" id="nama_pelapor" name="nama_pelapor">
13
14     <label for="lokasi">Lokasi Kejadian</label>
15     <input type="text" id="lokasi" name="lokasi">
16
17     <label for="tinggi_genangan">Tinggi Genangan Air (cm)</label>
18     <input type="number" id="tinggi_genangan" name="tinggi_genangan" min="1">
19
20     <button type="submit" class="btn">Kirim Laporan</button>
21   </form>
22 @endsection

```

Form memakai method POST dengan action route('laporan.store'). Directive @csrf menghasilkan token keamanan yang wajib ada pada setiap form POST di Laravel; kalau token ini dilewati, pengiriman form berhenti di halaman 419 Page Expired.

3.5 Membuat Komponen Blade

Komponen alert dibuat pada file resources/views/components/alert.blade.php dengan isi sebagai berikut:

```
1 @if ($type === 'error')
2 <div style="padding: 10px; border: 1px solid red; margin: 10px 0;">
3     @else
4     <div style="padding: 10px; border: 1px solid green; margin: 10px 0;">
5     @endif
6     <strong>{{ ucfirst($type) }}:</strong> {{ $message }}
7 </div>
8 </div>
```

File yang berada di folder resources/views/components otomatis dikenali sebagai komponen dengan tag berawalan x-. Atribut type dan message yang ditulis saat pemanggilan dibaca oleh komponen sebagai variabel \$type dan \$message. Kalau type berisi error maka blok @else dilewati dan kotak mendapat border merah, sedangkan tipe lain mendapat border hijau. Komponen dipanggil dari halaman konfirmasi seperti ini:

```
1 <x-alert type="success" message="Laporan banjir berhasil dikirim. Terima kasih atas laporan
Anda." />
```

3.6 Membuat Halaman Konfirmasi

Method store() menampilkan view resources/views/laporan/konfirmasi.blade.php begitu form dikirim. View ini juga mewarisi layout, lalu menampilkan komponen alert beserta data yang baru saja dikirim pengguna:

```

1  @extends('layouts.app')
2
3  @section('title', 'Konfirmasi Laporan')
4
5  @section('content')
6      <h1>Konfirmasi Laporan</h1>
7
8      <x-alert type="success" message="Laporan banjir berhasil dikirim. Terima kasih atas
        laporan Anda." />
9
10     <div class="detail">
11         <p>Berikut data laporan yang Anda kirim:</p>
12         <dl>
13             <dt>Nama Pelapor</dt>
14             <dd>{{ $laporan['nama_pelapor'] }}</dd>
15
16             <dt>Lokasi Kejadian</dt>
17             <dd>{{ $laporan['lokasi'] }}</dd>
18
19             <dt>Tinggi Genangan</dt>
20             <dd>{{ $laporan['tinggi_genangan'] }} cm</dd>
21         </dl>
22
23         <div class="actions">
24             <a href="{{ route('laporan.index') }}" class="btn">Lihat Daftar Laporan</a>
25             <a href="{{ route('laporan.create') }}" class="btn">Buat Laporan Lagi</a>
26         </div>
27     </div>
28 @endsection

```

Data \$laporan dikirim langsung dari controller ke view pada saat form diproses, tanpa penyimpanan ke mana pun. Jadi halaman konfirmasi hanya menampilkan kembali isi form yang tadi dikirim.

3.7 Membuat Partial dan Halaman Daftar

Kartu laporan dipisahkan ke file resources/views/partials/laporan-card.blade.php. Di dalam partial ini terdapat directive percabangan yang menentukan status genangan:

```

1 <article class="card">
2   <div class="card-head">
3     <strong>{{ $laporan['lokasi'] }}</strong>
4
5     @if ($laporan['tinggi_genangan'] < 30)
6       <span class="badge waspada">Waspada</span>
7     @elseif ($laporan['tinggi_genangan'] <= 70)
8       <span class="badge siaga">Siaga</span>
9     @else
10      <span class="badge awas">Bahaya</span>
11    @endif
12  </div>
13
14  <p><strong>Pelapor:</strong> {{ $laporan['nama_pelapor'] }}</p>
15  <p><strong>Tinggi genangan:</strong> {{ $laporan['tinggi_genangan'] }} cm</p>
16  <p><strong>Waktu laporan:</strong> {{ $laporan['waktu'] }}</p>
17 </article>

```

Genangan di bawah 30 cm berstatus Waspada, genangan 30 sampai 70 cm berstatus Siaga, dan genangan di atas 70 cm berstatus Bahaya. Angka 30 dan 70 ikut masuk rentang Siaga karena dipakai pada kondisi `<= 70`. Sementara itu, halaman daftar pada `resources/views/laporan/index.blade.php` cukup berisi perulangan yang memanggil partial:

```

1 @forelse ($laporans as $laporan)
2   @include('partials.laporan-card', ['laporan' => $laporan])
3 @empty
4   <p class="empty">Belum ada laporan banjir.</p>
5 @endforelse

```

Variabel `$laporan` tidak didefinisikan di dalam partial, melainkan dikirim dari view pemanggil lewat `@include` lengkap dengan nilainya. Karena itu detail kartu cukup ditulis sekali di partial, dan view utama tidak perlu mengulang kode yang sama untuk tiap laporan. Kalau array `$laporans` kosong, blok `@empty` menampilkan pesan bahwa belum ada laporan.

3.8 Menjalankan dan Menguji Aplikasi

Aplikasi dijalankan melalui development server Laravel dengan perintah `php artisan serve`, lalu alamat `http://127.0.0.1:8000` dibuka pada Google Chrome. Saya mengisi form, menekan tombol Kirim Laporan, lalu memeriksa halaman konfirmasi dan halaman daftar laporan

melalui menu navigasi. Ketiga halaman tampil tanpa galat, dan hasil pengujian ditunjukkan pada bagian berikutnya.

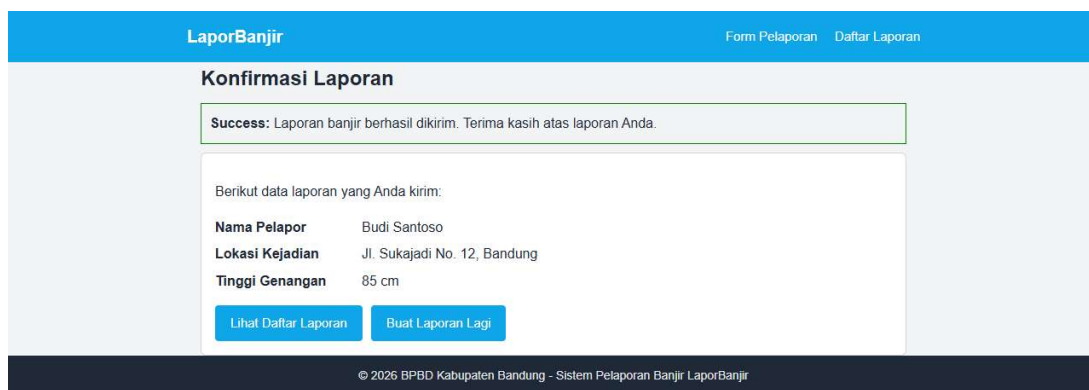
4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi LaporBanjir yang berjalan pada browser melalui php artisan serve.



Gambar 1. Halaman form pelaporan pada alamat /

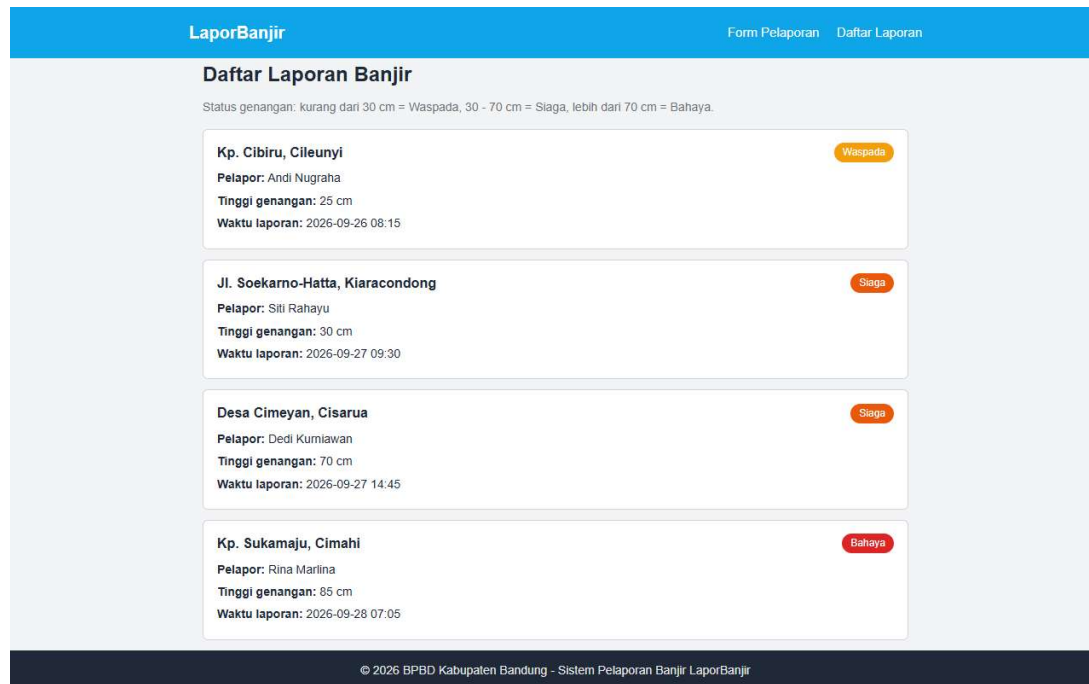
Gambar 1 menampilkan halaman form yang dihasilkan oleh route / pada method create(). Formulirnya memuat isian nama pelapor, lokasi kejadian, dan tinggi genangan air sesuai ketentuan tugas. Menu Form Pelaporan dan Daftar Laporan di header memakai named route, sedangkan footer menampilkan copyright BPBD Kabupaten Bandung.



Gambar 2. Halaman konfirmasi setelah form dikirim

Gambar 2 diambil setelah form diisi dengan nama Budi Santoso dan tombol Kirim Laporan ditekan. Data yang dikirim tampil kembali pada halaman konfirmasi, dan pesan berhasil

dikirim tampil di dalam komponen `<x-alert>` dengan border hijau karena atribut `type` berisi `success`. Halaman ini dibuat oleh method `store()` melalui route `POST /laporan`.



Gambar 3. Halaman daftar laporan beserta status genangan

Gambar 3 menampilkan empat laporan contoh pada alamat `/laporan`. Tiap kartu dirender oleh `partial laporan-card`, mulai dari laporan setinggi 25 cm berstatus Waspada, laporan 30 dan 70 cm berstatus Siaga, sampai laporan 85 cm berstatus Bahaya. Perulangan `@forelse` yang memanggil `partial` membuat keempat kartu tampil dengan bentuk yang sama.

5. Analisis dan Pembahasan

Tampilan aplikasi ini dibagi ke dalam tiga jenis file dengan alasan yang berbeda. Layout `app.blade.php` memegang bagian yang selalu sama pada setiap halaman, yaitu header, navigasi, footer, dan CSS, sehingga perubahan gaya cukup dilakukan di satu file dan langsung berlaku untuk ketiga halaman. Komponen `<x-alert>` dipakai untuk kotak pesan yang bentuknya sudah pasti dan hanya berbeda pada isi pesan serta tipenya, jadi komponen cukup dibuat satu kali di folder `components`. `Partial laporan-card` dipakai untuk satu kartu laporan yang dirender berulang di dalam `@forelse`; `partial` menerima variabel lewat `@include` sehingga view utama tidak perlu menulis detail kartu berkali-kali.

Pada ketentuan tugas, status untuk genangan di atas 70 cm bernama Awas. Pada aplikasi ini saya menulis status tersebut menjadi Bahaya, sementara rentang pengukurannya tetap sesuai ketentuan, yaitu di bawah 30 cm Waspada, 30 sampai 70 cm Siaga, dan di atas 70 cm Bahaya.

Sebelum memakai Blade, setiap halaman harus ditulis lengkap mulai dari tag DOCTYPE, tag head, navigasi, sampai footer seperti pada Modul 2. Setelah layout dipakai, ketiga halaman hanya berisi judul dan isinya saja karena kerangka halaman diambil dari `@yield`. Tampilan di browser juga tetap sama setelah kartu laporan dipindah ke partial, karena pemisahan file hanya mengatur struktur kode dan tidak mengubah hasil render.

Kendala yang saya temui selama pengerjaan ada dua. Pertama, halaman sempat gagal dengan galat syntax error, unexpected end of file, expecting elseif or else or endif. Penyebabnya adalah teks `<x-alert>` yang ikut tertulis di dalam komentar CSS pada file layout, karena Blade menganggap teks tersebut sebagai pemanggilan komponen lalu mengubahnya menjadi kode PHP yang tidak lengkap. Setelah komentar diubah menjadi teks biasa, halaman kembali normal. Kedua, development server php artisan serve beberapa kali berhenti sendiri di tengah pengujian, sehingga perintahnya harus dijalankan ulang dulu sebelum pengujian dilanjutkan.

6. Kesimpulan

Berdasarkan praktikum tugas Modul 3 ini, sistem pelaporan banjir LaporBanjir berhasil dibuat dengan ketiga halamannya, yaitu form pelaporan, konfirmasi, dan daftar laporan. Ketiganya dapat diakses melalui route yang mengarah ke method controller, dan seluruh halaman tampil seragam karena mewarisi layout `app.blade.php` lewat `@extends` dan `@section`.

Directive Blade juga berjalan sesuai kebutuhan: `@if`, `@elseif`, dan `@else` menentukan status tiap laporan, `@forelse` menampilkan daftar laporan lengkap dengan pesan kosongnya, komponen `<x-alert>` menampilkan pesan laporan berhasil dikirim, dan partial dipanggil dengan `@include` untuk menampilkan tiap laporan dalam bentuk kartu. Melalui praktikum ini saya jadi paham kapan memakai layout, kapan memakai komponen, dan kapan memakai partial, serta bagaimana route, controller, dan view bekerja bersama dalam pola MVC.

7. Link Repository Github

<https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/praktikum3-ta>